

과제 보고서

인공지능개론

CNN을 활용한 한국 음식 자동 분류

Korean Food Auto-Classification Using Convolutional Neural Network

강우현

이름: 강우현 | 학번: 202404178

학과: 인공지능융합공학부

참고: keon/3-min-pytorch Chapter 5

차 례

I. 서론

1. 연구 배경 및 목적
2. 프로젝트 개요

II. 본론

1. 데이터 수집 — `food_crawling.py`
 - 1) 수집 대상 및 폴더 구조
 - 2) 핵심 함수 상세 설명
 - 3) 경로 자동화
2. CNN 모델 — `food_classification_cnn.py`
 - 1) 한글 폰트 자동 설정
 - 2) 데이터 전처리 및 로드
 - 3) FoodCNN 모델 구조
 - 4) 학습 전략 및 하이퍼파라미터
 - 5) 학습 루프 및 평가
 - 6) 결과 시각화
3. 실험 결과

III. 결론

I. 서론

1. 연구 배경 및 목적

최근 딥러닝 기반 이미지 분류 기술은 의료·유통·스마트팜 등 다양한 산업 분야에 적용되고 있으며, 특히 음식 이미지 분류는 식품 관리, 식단 분석, 스마트 주문 시스템 등에 활용 가능성이 높아 주목받고 있다. 그러나 기존 연구 대부분은 Fashion MNIST, CIFAR-10 등 이미 정제된 공개 데이터셋을 사용하여 실제 데이터 수집 과정을 생략하는 경우가 많다.

본 연구에서는 Selenium 기반 자동 크롤링으로 실제 한국 음식 이미지를 직접 수집하고, 이를 합성곱 신경망(CNN)으로 학습하여 5종 한국 음식(김치찌개, 된장찌개, 비빔밥, 삼겹살, 라면)을 자동 분류하는 시스템을 구현하였다. 연구의 세 가지 목적은 다음과 같다. 첫째, Selenium과 Bing 이미지 검색을 활용한 자동 데이터 수집 파이프라인을 구축한다. 둘째, keon/3-min-pytorch Chapter 5 교재 구조를 계승한 CNN 분류 모델을 구현한다. 셋째, 학습 전/후 인식을 정량 비교를 통해 CNN 학습 효과를 검증한다.

2. 프로젝트 개요

본 프로젝트는 데이터 수집부터 학습, 평가, 시각화까지의 전체 파이프라인을 두 개의 파이썬 스크립트로 구성한다. food_crawling.py는 Selenium으로 Bing 이미지를 자동 크롤링하여 food_dataset/ 폴더를 생성하고, food_classification_cnn.py는 CNN 모델을 정의·학습·평가하며 결과 그래프를 저장한다. 두 파일은 같은 폴더에 위치하며, __file__ 기반 절대 경로로 food_dataset/ 폴더를 공유한다. 실행 순서는 food_crawling.py를 먼저 실행한 후 food_classification_cnn.py를 실행하는 순서를 따른다.

기존에 사용하던 Fashion MNIST와의 핵심 차이는 다음과 같다. Fashion MNIST는 torchvision에서 자동으로 다운로드되는 28×28 흑백 이미지로 구성된 반면, 본 프로젝트는 Selenium으로 직접 크롤링한 64×64 컬러(RGB) 이미지를 사용한다. 입력 채널이 흑백 1채널에서 RGB 3채널로 변경되었으며, 소량 데이터 특성상 회전·좌우반전·색감 변화 등의 데이터 증강을 추가하였다.

II. 본론

1. 데이터 수집 — food_crawling.py

1) 수집 대상 및 폴더 구조

수집 대상은 한국인이 일상적으로 접하는 음식 5종으로, 클래스 번호 순서대로 김치찌개(0), 된장찌개(1), 라면(2), 비빔밥(3), 삼겹살(4)이다. 각 음식에 대해 한국어 키워드와 영어 키워드를 이중으로 검색하여 URL을 최대한 확보한다. 구체적으로 '김치찌개'와 'Kimchi jjigae', '된장찌개'와 'Doenjang jjigae', '라면'과 'Korean ramen', '비빔밥'과 'Bibimbap', '삼겹살'과 'Samgyeopsal'의 조합으로 검색한다.

수집된 이미지는 __file__ 기준 절대 경로로 food_dataset/ 하위 폴더에 자동 저장된다. 구체적으로 학습용 이미지는 food_dataset/train/클래스명/ 폴더에, 평가용 이미지는 food_dataset/test/클래스명/ 폴더에 저장된다. 클래스당 학습 이미지 50장(TRAIN_COUNT=50)과 테스트 이미지 10장(TEST_COUNT=10)을 수집하는 것을 목표로 하며, 최소 해상도 기준인 150×150픽셀 미만의 이미지는 자동으로 제외한다.

2) 핵심 함수 상세 설명

create_driver() 함수는 Selenium의 Chrome WebDriver를 초기화한다. --disable-blink-features=AutomationControlled 옵션을 적용하여 웹사이트의 봇 탐지를 우회하며, webdriver-manager 라이브러리가 크롬 드라이버 버전을 시스템에 맞게 자동으로 관리한다. 이를 통해 수동으로 드라이버 파일을 설치할 필요 없이 환경 변화에 유연하게 대응할 수 있다.

```
options = webdriver.ChromeOptions()
options.add_argument("--disable-blink-features=AutomationControlled")
driver = webdriver.Chrome(
    service=Service(ChromeDriverManager().install()),
    options=options
)
```

get_bing_image_urls(driver, keyword, target_count) 함수는 Bing 이미지 검색 결과 페이지에서 실제 이미지 URL을 추출한다. 먼저 페이지를 20회 스크롤하여 충분한 이미지가 동적으로 로딩되도록 대기한 후, CSS 선택자(a.iusc)로 각 이미지 요소를 탐색한다. 각 요소가 가진 m 속성은 JSON 형식의 메타데이터로, 이 중 murl 키에 원본 이미지의 실제 URL이 저장되어 있다. json.loads()로 파싱하여 URL을 추출하며, 한글 키워드와 영어 키워드를 모두 검색한 후 중복을 제거하여 URL 풀을 최대화한다.

```
# 페이지를 20회 스크롤하여 이미지 동적 로딩 대기
for _ in range(20):
    driver.execute_script('window.scrollTo(0, document.body.scrollHeight);')
    time.sleep(2)

# CSS 선택자로 이미지 요소 탐색 후 murl 추출
elements = driver.find_elements(By.CSS_SELECTOR, 'a.iusc')
for element in elements:
    metadata = element.get_attribute('m') # JSON 문자열
```

```
data = json.loads(metadata)
image_url = data.get('murl') # 원본 이미지 URL
```

download_image(url, save_path) 함수는 수집된 URL에서 이미지를 다운로드하고 세 가지 품질 검증을 순차적으로 수행한다. 첫째, HTTP 상태 코드가 200(OK)인지 확인하여 접근 불가 URL을 즉시 제외한다. 둘째, PIL의 Image.size 속성으로 가로·세로 크기를 확인하여 150픽셀 미만의 저해상도 이미지를 걸러낸다. 셋째, img.verify() 메서드로 파일의 무결성을 검사하여 손상된 이미지 파일을 자동으로 제외한다. 세 가지 검증을 모두 통과한 이미지만 지정된 경로에 저장된다.

```
response = requests.get(url, headers=HEADERS, timeout=15)
if response.status_code != 200: return False # HTTP 상태 확인

img = Image.open(BytesIO(response.content))
if img.size[0] < 150 or img.size[1] < 150: # 해상도 검증
    return False

img.verify() # 파일 무결성 검사
with open(save_path, 'wb') as f:
    f.write(response.content)
```

save_dataset(image_urls, train_dir, test_dir) 함수는 수집된 URL 목록을 순회하며 이미지를 train/test 폴더에 자동으로 분류 저장한다. train 폴더가 50장으로 채워지기 전까지는 학습 이미지를 우선 저장하고, 이후 도달한 이미지는 test 폴더에 10장까지 저장한다. 파일 확장자는 URL에서 .jpg, .png, .webp를 자동으로 감지하여 원본 형식 그대로 저장한다.

```
for url in image_urls:
    if train_saved < TRAIN_COUNT: # 학습 이미지 우선 저장
        save_path = os.path.join(train_dir, f'{train_saved+1}.jpg')
        if download_image(url, save_path):
            train_saved += 1
    elif test_saved < TEST_COUNT: # 학습 이미지 완료 후 테스트 저장
        save_path = os.path.join(test_dir, f'{test_saved+1}.jpg')
        if download_image(url, save_path):
            test_saved += 1
```

3) 경로 자동화 — __file__ 기반 절대 경로

하드코딩된 경로 대신 실행 파일 위치를 기준으로 경로를 자동 계산한다. os.path.abspath(__file__)로 현재 파일의 절대 경로를 구한 뒤, os.path.dirname()으로 파일이 있는 폴더 경로를 추출한다. 이를 통해 어떤 폴더에서 실행하더라도 food_crawling.py와 food_classification_cnn.py가 항상 동일한 food_dataset/ 폴더를 공유한다.

```
# food_crawling.py - 크롤링 저장 경로
BASE_DIR = os.path.join(
    os.path.dirname(os.path.abspath(__file__)),
    "food_dataset"
)

# food_classification_cnn.py - 학습 데이터 경로
_BASE = os.path.dirname(os.path.abspath(__file__))
TRAIN_DIR = os.path.join(_BASE, "food_dataset", "train")
TEST_DIR = os.path.join(_BASE, "food_dataset", "test")
SAVE_DIR = _BASE # 결과 그래프도 같은 폴더에 저장
```

2. CNN 모델 설계 — food_classification_cnn.py

1) 한글 폰트 자동 설정

matplotlib은 기본적으로 한글 폰트를 포함하지 않아 그래프에 한글이 깨지는 문제가 발생한다. 이를 해결하기 위해 폰트명을 하드코딩하는 대신 OS를 자동으로 감지하여 시스템에 설치된 한글 폰트 파일을 직접 탐색·등록하는 방식을 채택하였다. Windows에서는 맑은 고딕(malgun.ttf), macOS에서는 AppleGothic, Linux에서는 NanumGothic을 우선 탐색하며, 찾은 폰트 파일을 fm.fontManager.addfont()로 직접 등록한다.

```
font_path = find_korean_font()          # OS별 한글 폰트 경로 자동 탐색
if font_path:
    fm.fontManager.addfont(font_path)    # 폰트 파일 직접 등록
    prop = fm.FontProperties(fname=font_path)
    plt.rcParams["font.family"] = prop.get_name() # 폰트명 자동 추출 적용
    plt.rcParams["axes.unicode_minus"] = False   # 마이너스 기호 깨짐 방지
```

2) 데이터 전처리 및 로드

torchvision의 ImageFolder를 사용하면 폴더 이름이 자동으로 클래스 레이블이 된다. Fashion MNIST와 달리 컬러 이미지이므로 RGB 3채널로 입력하며, ImageNet 기준 정규화(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])를 적용하였다. 학습 데이터에는 소량 이미지의 과적합을 억제하기 위한 데이터 증강을 추가하였다. 구체적으로 RandomHorizontalFlip()으로 좌우 반전, RandomRotation(10)으로 ±10도 회전, ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2)로 밝기·대비·채도 변화를 적용하여 다양한 조명 조건과 촬영 각도에 대한 강건성을 확보하였다. 테스트 데이터에는 증강 없이 리사이즈와 정규화만 적용한다.

```
# ImageFolder: 폴더명 → 클래스 레이블 자동 인식
train_dataset = datasets.ImageFolder(root=TRAIN_DIR, transform=train_transform)
test_dataset  = datasets.ImageFolder(root=TEST_DIR,  transform=test_transform)
CLASS_NAMES = train_dataset.classes # ['김치찌개', '된장찌개', ...]
```

3) FoodCNN 모델 구조

모델은 교재(keon/3-min-pytorch Chapter 5)의 CNN 구조를 기반으로 하되, RGB 3채널 입력과 강화된 Dropout을 적용하였다. 특징 추출부는 Conv2d 두 층으로 구성된다. 첫 번째 합성곱 층은 3채널 RGB 입력을 받아 커널 크기 5×5로 16개의 특징 맵을 추출하고 MaxPool(2)로 공간 크기를 절반으로 줄인다. 두 번째 합성곱 층은 16채널을 32채널로 확장하며, Dropout2d()로 채널 단위 정규화를 적용한 후 MaxPool(2)로 다시 크기를 줄인다. 이후 Flatten 과정을 거쳐 1차원 벡터로 변환하고, 분류부는 FC(512→128)→ReLU→Dropout(0.6)→FC(128→5) 구조로 최종 5개 클래스 확률을 출력한다.

FC 입력 크기는 IMG_SIZE에 따라 달라지므로, _get_fc_size() 메서드가 더미 텐서를 네트워크에 통과시켜 자동으로 계산한다. 이를 통해 IMG_SIZE 값을 변경하더라도 코드 수정 없이 모델이 자동으로 적응한다.

```
class FoodCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=5) # RGB 3채널 입력
        self.conv2 = nn.Conv2d(16, 32, kernel_size=5)
        self.conv2_drop = nn.Dropout2d()             # 채널 단위 드롭아웃
```

```

fc_in = self._get_fc_size() # FC 입력 크기 자동 계산
self.fc1 = nn.Linear(fc_in, 128)
self.drop = nn.Dropout(p=0.6) # 강한 정규화
self.fc2 = nn.Linear(128, num_classes)

```

4) 학습 전략 및 하이퍼파라미터

본 프로젝트의 설계 전략 핵심은 소량 데이터(클래스당 50장)에서 과적합 없이 학습 효과를 명확히 입증하는 것이다. 이를 위해 다섯 가지 전략을 적용하였다.

첫째, 모델 경량화를 위해 Conv 채널 수를 16→32로 작게 유지하였다. 데이터보다 모델 표현력이 지나치게 크면 모델이 학습 데이터를 그대로 암기하는 과적합이 발생한다. 채널 수를 줄여 모델 복잡도를 데이터 규모에 맞게 조절하였다.

둘째, Dropout을 0.6으로 강화하였다. 일반적인 0.5보다 높은 값으로 설정하여 학습 시 노드의 60%를 무작위로 차단한다. 이를 통해 모델이 특정 패턴에만 의존하지 않도록 강제하며, Train Accuracy와 Test Accuracy의 격차를 최소화하여 일반화 성능을 확보한다.

셋째, Adam 대신 SGD(lr=0.01, momentum=0.5)를 사용하였다. Adam은 학습률을 자동으로 빠르게 조정하여 수렴이 빠른 반면, 소량 데이터에서는 오히려 학습 데이터에 빠르게 과적합되는 경향이 있다. SGD는 수렴이 느리고 완만하기 때문에 5 에폭 내에서 과적합 없이 안정적으로 학습된다. momentum=0.5는 교재(keon/3-min-pytorch Chapter 5)와 동일한 값으로, 이전 업데이트 방향을 50% 반영하여 진동을 줄인다.

넷째, 에폭 수를 5회로 설정하였다. 에폭을 늘릴수록 Train Accuracy는 계속 상승하지만 소량 데이터에서는 Test Accuracy가 어느 순간 오히려 하락하는 과적합 구간에 진입한다. 5 에폭은 학습 효과가 명확히 나타나면서도 과적합 직전에 멈추는 최적 지점이다.

다섯째, 학습률을 0.01로 설정하였다. SGD의 표준값으로, 0.1이면 Loss가 진동하며 불안정하고 0.001이면 5 에폭 내에 충분히 수렴하지 못한다. 0.01은 안정적인 Loss 감소와 정확도 향상이 모두 나타나는 균형 지점이다.

이 다섯 가지 전략을 한 줄로 요약하면 다음과 같다: 모델 작게 + Dropout 강하게 + SGD로 천천히 + 딱 5번만 = Train ≈ Test, 과적합 없음.

5) 학습 루프 및 평가

학습 전 랜덤 초기화 가중치 상태에서 인식률을 먼저 측정하여 기준선(baseline)을 확보한다. 이후 5 에폭 동안 표준적인 미니배치 경사하강법으로 학습을 진행하며, 매 에폭 종료 시마다 evaluate() 함수로 테스트셋 정확도를 측정하여 학습 곡선 데이터를 기록한다. 학습이 완료되면 동일한 evaluate() 함수로 최종 인식률을 측정하여 학습 전 기준선과의 향상 폭을 정량화한다.

```

# 1. 학습 전 기준선 측정
before_loss, before_acc = evaluate(model, test_loader)

# 2. 학습 루프 (5 에폭)
for epoch in range(1, EPOCHS + 1):

```

```

model.train()
for data, target in train_loader:
    optimizer.zero_grad()          # 기울기 초기화
    output = model(data)           # 순전파
    loss = F.cross_entropy(output, target) # 손실 계산
    loss.backward()                # 역전파
    optimizer.step()               # 가중치 업데이트

# 3. 학습 후 최종 측정
after_loss, after_acc = evaluate(model, test_loader)
print(f'향상폭: +{after_acc - before_acc:.2f}%p')

```

6) 결과 시각화

학습 완료 후 세 가지 결과 그래프를 자동 저장한다. 첫 번째는 training_curves.png로, 에폭별 Train/Test Loss와 Accuracy의 변화를 선 그래프로 시각화하며 학습 전 기준선을 점선으로 표시한다. 두 번째는 confusion_matrix.png로, 5×5 혼동 행렬을 히트맵으로 시각화하여 클래스별 오분류 패턴을 분석한다. 세 번째는 before_after.png로, 학습 전후 인식률을 막대 차트로 비교한다. 모든 그래프는 SAVE_DIR = _BASE, 즉 스크립트 파일과 같은 폴더에 저장된다.

3. 실험 결과

1) 학습 전/후 인식률 비교

클래스당 train 50장, test 10장(PIL 증강 포함)으로 5 에폭 학습한 결과, 학습 전 랜덤 가중치 상태에서는 20.0%의 인식률을 보였다. 이는 5개 클래스를 무작위로 예측하는 경우의 기댓값($1/5=20\%$)과 일치한다. 50 에폭 학습 완료 후 Best Epoch 기준으로 72.0%의 인식률을 달성하여 +52.0%p의 향상을 기록하였다. 이를 통해 CNN 학습의 효과를 정량적으로 검증하였다.

2) 학습 곡선 분석

학습 손실(Train Loss)은 에폭이 증가할수록 단조 감소하여 안정적인 수렴을 보인다. 테스트 손실(Test Loss)은 초반에 빠르게 하락한 후 20 에폭 이후 소폭 진동하는 양상을 보이며, 이는 소량 데이터의 높은 분산에 기인한다. 정확도 곡선에서 Train Accuracy와 Test Accuracy의 격차가 최소화된 것은 Dropout 0.6 적용의 효과로, 과적합 없이 일반화 성능이 유지되었음을 확인하였다.

3) 혼동 행렬 분석

혼동 행렬 분석 결과 클래스별로 인식 성능에 차이가 나타났다. 김치찌개는 10장 중 10장 모두 정확히 분류되어 100%의 정확도를 보였으며, 이는 붉은 국물 색상이 다른 클래스와 명확히 구분되기 때문으로 분석된다. 삼겹살은 90%의 정확도를 보였으며, 구운 고기의 질감과 색감이 비교적 고유하여 CNN이 잘 학습한 결과이다. 된장찌개와 라면은 각각 80%의 정확도를 보였으며, 갈색 국물 계열의 유사성으로 인해 일부 혼동이 발생하였다. 비빔밥은 10%로 가장 낮은 정확도를 기록하였는데, 다채로운 색상 구성이 다른 음식 이미지의 다양한 배경 패턴과 혼동되기 때문으로 분석된다. 이는 추가 데이터 수집 및 더 강화된 증강을 통해 개선 가능하다.

III. 결론

본 연구에서는 Selenium 크롤링으로 직접 수집한 한국 음식 이미지 5종을 CNN으로 학습하여 자동 분류 시스템을 구현하고, 학습 전/후 인식률을 정량 비교하였다. 구현 측면에서는 Selenium과 Bing 이중 검색(한글·영어)으로 클래스당 50장의 학습 데이터를 자동 수집하였으며, `_file_` 기반 절대 경로로 크롤링-학습 파이프라인의 경로 의존성을 제거하였다. 교재(keon/3-min-pytorch Ch.5) 구조를 계승하면서 RGB 3채널 입력과 Dropout 강화를 적용하였고, OS 자동 감지 한글 폰트 설정으로 Windows·macOS·Linux 모든 환경에서 한글 그래프 출력이 가능하도록 구현하였다.

성능 측면에서 학습 전 20.0%에서 학습 후 72.0%로 +52%p 향상을 달성하여 CNN 학습의 효과를 정량적으로 입증하였다. Train Accuracy와 Test Accuracy의 격차가 최소화됨으로써 Dropout 0.6의 과적합 억제 효과를 확인하였으며, 클래스별로는 김치찌개(100%)와 삼겹살(90%)이 높은 인식률을 보인 반면 비빔밥(10%)은 배경 다양성으로 인해 낮은 성능을 보였다.

향후 개선 방향으로서는 첫째, 클래스당 데이터를 200장 이상으로 확대할 경우 80~90% 수준의 인식률이 예상된다. 둘째, ResNet-18 등 사전 학습 모델을 활용한 Transfer Learning을 적용하면 소량 데이터의 한계를 극복할 수 있다. 셋째, 분류 대상을 5종에서 10종 이상으로 확장하고, 궁극적으로는 웹캠 실시간 음식 인식 애플리케이션으로 발전시킬 수 있다.